

Reviewer Pack — Riemann Hypothesis and Read-Once Branching Program Size

Entry be2c11ba6553 · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-05-29 08:01:20 UTC

Riemann Hypothesis and Read-Once Branching Program Size

Entry ID: be2c11ba6553 **Verdict:** INCONCLUSIVE **Recorded:** 2026-05-29 08:01:20 UTC

1. Conjecture

Field A (mathematical branch): Number Theory (Riemann Zeta Function) **Field B** (complexity object): Boolean Function Complexity (Read-Once Branching Program Size)

Statement:

For all boolean functions f , the Riemann zeta function $\zeta(s)$ at $s=1/2$ has non-trivial zeros if and only if there exists a read-once branching program computing f with size $O(n^{3/4})$.

Rationale (proposer's reasoning):

The Riemann Hypothesis in number theory is closely related to the distribution of prime numbers, while boolean functions and their complexity are at the heart of computation. A connection between these two fields could reveal a fundamental principle linking computational hardness and arithmetic properties.

Taxonomy category: TROPICAL_FOURIER_ANALYSIS (status at proposal time:)

2. Pre-registration (Popper-style)

Hash (SHA-256 prefix): 99acb40b902adbe3

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

Acceptance criterion:

The Riemann Hypothesis is supported if, across 30 different seeds, at least 80% of the generated boolean functions f with $n=40$ have a corresponding read-once branching program size of $O(n^{3/4})$ and the Riemann zeta function $\zeta(s)$ at $s=1/2$ exhibits non-trivial zeros. The hypothesis is falsified if less than 80% of the functions meet this condition or if any seed results in a metric > 10 for the branching program size.

3. Barrier filter (F1)

Final: PASS

Barrier	Verdict	Confidence	LLM ₁	LLM ₂
RELATIVIZATION	UNCERTAIN	1.00	UNCERTAIN	HITS
NATURAL_PROOFS	UNCERTAIN	0.95	HITS	UNCERTAIN
ALGEBRIZATION	UNCERTAIN	0.80	UNCERTAIN	HITS
KARP_LIPTON	SAFE	0.50	UNCERTAIN	UNCERTAIN

4. Novelty audit

Verdict: NOVEL against 0 hits across arXiv + Semantic Scholar + ECCC

Search queries (3): - "Riemann Hypothesis" AND "read-once branching program" AND "size $O(n^{3/4})$ " - "zeta function" AND "Boolean function complexity" AND "branching program" - "non-trivial zeros Riemann zeta function" AND "computing Boolean functions" AND "read-once branching programs"

5. Empirical test harness

Multi-seed protocol: 5 seeds (11, 23, 37, 53, 71)

Sandbox: isolated subprocess, stdlib-only, ≤ 90 s wall time per cycle **Execution:** rc=124, elapsed=240.4s

5.1 Generated Python source

```
import random
import math

def run_trial(seed: int) -> dict:
    random.seed(seed)

    def zeta(s):
        if s == 1:
            return float('inf')
        result = 0
        for n in range(1, 10000): # Avoid divergence by limiting the sum
            result += 1 / (n ** s)
        return result

    def is_prime(n):
        if n <= 1:
            return False
        if n <= 3:
            return True
        if n % 2 == 0 or n % 3 == 0:
            return False
        i = 5
        while i * i <= n:
            if n % i == 0 or n % (i + 2) == 0:
                return False
            i += 6
        return True

    def generate_boolean_function(n):
        return [random.choice([0, 1]) for _ in range(2 ** n)]

    def branching_program_size(f, n):
```

```

    if len(f) != 2 ** n:
        return None
    size = 0
    for i in range(n):
        size += 1 + max(branching_program_size(f[:2**(n-i-1)]), branching_program_size(f[2**(n-i-1):]))
    return size

def riemann_hypothesis_test():
    non_trivial_zeros = 0
    for _ in range(30):
        f = generate_boolean_function(40)
        zeta_value = zeta(0.5)
        if not is_prime(int(zeta_value)):
            continue
        bp_size = branching_program_size(f, 40)
        if bp_size is None or bp_size > 2 * (40 ** (3/4)):
            non_trivial_zeros += 1
    return non_trivial_zeros

non_trivial_zeros = riemann_hypothesis_test()

metric_value = non_trivial_zeros / 30
conjecture_holds = metric_value >= 0.8
counterexample = "" if conjecture_holds else "mapping_undefined"

return {
    "metric_name": "Non-trivial Zeros",
    "metric_value": metric_value,
    "instances_tested": 30,
    "conjecture_holds": conjecture_holds,
    "counterexample": counterexample
}

if __name__ == "__main__":
    seeds = [int(x) for x in sys.argv[1:]] or [2, 3, 5, 7, 11, 13, 17, 19, 23, 29] + list(range(31, 100))
    results = []

    for seed in seeds:
        result = run_trial(seed)
        print(f"TRIAL: {result}")
        results.append(result)

    mean_value = sum(r["metric_value"] for r in results) / len(results)
    std_value = math.sqrt(sum((r["metric_value"] - mean_value) ** 2 for r in results) / len(results))
    support_fraction = sum(1 for r in results if r["conjecture_holds"]) / len(results)

    if all(r["conjecture_holds"] for r in results):
        print(f"RESULT: SUPPORTED mean={mean_value} std={std_value} support_fraction={support_fraction}")
    elif support_fraction >= 0.8:
        print(f"RESULT: SUPPORTED mean={mean_value} std={std_value} support_fraction={support_fraction}")
    else:
        first_failing_seed = next(r["seed"] for r in results if not r["conjecture_holds"])
        print(f"RESULT: FALSIFIED counterexample=\"mapping_undefined\" first_failing_seed={first_failing_seed}")

```

6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

7. Test stdout (last 2KB)

TIMEOUT after 240s

8. Critic adversarial review

Critic verdict: CHALLENGE

Critic reasoning:

skipped: test crashed before producing data

9. Final verdict & safety rail

Verdict: INCONCLUSIVE

Reasoning:

The test timed out before producing data, which prevents us from verifying if the Riemann Hypothesis is supported or falsified according to the pre-registered criteria. | next: NONE

11. Audit log (LLM calls)

Total LLM calls: 9

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
1	propose	ollama_remote	glm4:latest	0	0	11048
2	preregistration	ollama_remote	glm4:latest	0	0	6068
3	novelty	ollama_remote	glm4:latest	0	0	4949
4	novelty	ollama_remote	glm4:latest	0	0	11044
5	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	12712
6	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	8635
7	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	10268
8	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	9719
9	judge	ollama_remote	glm4:latest	0	0	10521

Totals: 0 input tokens, 0 output tokens, ~\$0.0000 cost, 84964 ms total latency. Provider mix: {'ollama_remote': 9}

(full prompt+response transcripts available in `research/audit/be2c11ba6553.jsonl`)

12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice
3. The full LLM transcripts are in `research/audit/be2c11ba6553.jsonl` for verification of provenance
4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

Sandbox file: `research/pvsnp_sandbox/test_*.py` (per-cycle)

Replay tarball: `research/replay/be2c11ba6553.tar.gz` (if generated)